PRACTICE EXAM

Sample Questions

for Claude Architect Exam

Dump 2

66 Questions • Agentic Systems • Context & Escalation • Batch API • Extraction • Tool Design • MCP • AI Fluency

Topic Index & Weightage

TOPIC	Q RANGE	COUNT	WEIGHT
Context & Escalation (Agentic)	Q1–Q12	12	18%
Batch API & Extraction	Q13–Q20	8	12%
Tool Design	Q21–Q34	14	21%
MCP Design Patterns	Q35–Q46	12	18%
Agentic Systems & Code Agents	Q47–Q52	6	12%
AI Fluency Framework	Q53–Q66	14	19%

💡 *Higher-weighted topics = more exam questions. Prioritize Tool Design, MCP, and Agentic Systems for maximum score impact.*

SECTION

🧠 Context & Escalation

Managing conversation state, tool failures, and human routing

1

CONTEXT MANAGEMENT

1 Your customer-service agent has completed 18 turns helping with an account upgrade. Each turn includes a full get_account_details response with 55 fields. At turn 19, the customer asks about their current plan tier. The context window is 70% full. What is the most effective approach to continue the conversation?

A **Summarize the account details into a compact structured object retaining only plan tier, billing cycle, and usage, then continue with the new question.** ✓ CORRECT

B Truncate the oldest 8 turns from the conversation history to free up space — an approach that treats the symptom rather than the cause.

C Start a fresh session and ask the customer to re-explain their situation — a tempting shortcut that skips the underlying fix.

D Call get_account_details again and use only the latest response, discarding prior ones — a choice that looks simpler but shifts the problem elsewhere.

💡 Answer A — Selective summarization compresses the bloated tool outputs into a compact, structured form that preserves the information needed for the next action. Truncating turns (B) loses conversational context; a fresh session (C) degrades CX; re-calling the tool (D) wastes tokens on the same verbose data.

2

ESCALATION HANDOFF

2 A customer disputes a charge of $1,240 from 60 days ago, which your policy covers only up to 45 days. After 12 turns, you've confirmed the charge, the policy limit, and the customer's bank statement. The customer becomes angry. You must escalate. What context is most valuable to pass to the human agent?

A The last three turns of the conversation verbatim — a workaround that adds complexity without improving reliability.

B A structured summary: charge amount, charge date, policy window, confirmed eligibility status, and the customer's emotional state. ✓ CORRECT

C The customer's original complaint message only — a quick fix that tends to resurface the same failure later.

D All 12 turns plus all tool call results — an option that holds only under assumptions this scenario breaks.

💡 Answer B — A structured handoff surfaces every fact the human agent needs without cognitive overload. Raw transcripts (D) are too long; the original complaint alone (C) discards investigative findings; 3-turn excerpts (A) are arbitrary.

3

STATEFUL CONVERSATION

3 Your password-reset agent verifies identity through 4 sequential steps. In QA testing, after the customer completes step 2, the agent asks for their date of birth again — which was already captured in step 1. The API is being called correctly each time. What is the most likely root cause?

A The system prompt lacks instructions to carry forward answers from prior steps.

B Claude's context window was exceeded during step 2 and the earlier turns were dropped.

C **The conversation history is not being included in the API request for subsequent steps.** ✓ CORRECT

D The identity-verification tool clears its local cache after each successful step.

💡 Answer C — Claude is stateless. Without the full conversation history in each request, prior turns are invisible to the model. There is no automatic memory or carry-forward; the system prompt (A) cannot substitute for missing history.

4

SESSION RESUMPTION

4 A customer paused a 28-turn return session 6 hours ago. Tool calls from that session show item status 'IN_TRANSIT'. You resume the session with the full history. Your first fresh tool call returns status 'DELIVERED'. The agent still refers to the item as in transit. What went wrong?

A The model's attention weights favour early-context tokens over late ones — a route that optimizes effort over correctness here.

B The model cached the prior tool response and did not register the new result — a plausible-sounding move that misreads the root cause.

C The fresh tool call must be wrapped in a special 'refresh' message type to override stale results.

D With 28 turns of history already in context, the model gave more weight to the earlier tool result than to the fresh one at the end. ✓ CORRECT

💡 Answer D — Long histories with conflicting tool results can cause the model to anchor on earlier data. The reliable fix is to start a fresh session with a structured summary plus fresh tool calls, eliminating the stale data entirely.

5

AGENTIC LOOP MECHANICS

5 Your agent calls check_inventory and receives: {item_id: 'SKU-449', qty: 3, warehouse: 'DFW', reorder_threshold: 5}. How does the agent determine whether to call place_reorder or notify_manager next?

A **The tool result is appended to the conversation and the model generates the next action by reasoning over the full context.** ✓ CORRECT

B An orchestration layer reads the qty field and routes to the appropriate tool automatically.

C The agent follows a pre-built decision tree embedded in the tool's response metadata.

D The agent executes the tool sequence pre-planned at the start of the workflow — an option that scales poorly as the system grows.

💡 Answer A — There is no separate routing layer. Tool results become part of the context and the model reasons about the next action. The model here would note qty (3) < reorder_threshold (5) and decide accordingly.

6

ERROR HANDLING

6 Your agent handles shipping queries. get_shipment_status returns the same generic error string for three distinct situations: carrier API timeout, invalid tracking number, and shipment not yet in carrier system. Logs show inconsistent retry behavior. What is the most effective fix?

A Add a retry_shipment_lookup tool the agent can call to determine the retry strategy — a step that appears safe but reduces observability.

B **Enrich tool error responses with an errorCode enum (TIMEOUT / INVALID_ID / NOT_YET_SCANNED), an isRetryable boolean, and a plain-language description.** ✓ CORRECT

C Include few-shot examples showing how to parse the generic error string and choose an action — a tactic that trades long-term reliability for short-term ease.

D Implement exponential backoff inside the tool for all error types — a change that leaves the real failure mode unaddressed.

💡 Answer B — Typed error metadata gives the model unambiguous signals. A second tool call (A) adds latency; few-shot string parsing (C) is fragile; blanket retries (D) waste turns on non-retriable errors.

7

PARTIAL FAILURE RECOVERY

7 Your travel booking agent retrieves flights and seat availability, but when it calls confirm_booking, the payment gateway times out. The customer's credit card has been pre-authorized. What response approach best balances transparency and customer experience?

A Tell the customer the booking is confirmed and follow up by email if the gateway fails permanently.

B Immediately escalate to a human agent — an easy-to-reach answer that doesn't survive closer scrutiny.

C **Explain what succeeded (pre-authorization), acknowledge the timeout, inform the customer the booking is pending, and offer to retry or escalate.** ✓ CORRECT

D Retry confirm_booking automatically up to 5 times before responding to the customer — a fix that addresses configuration when the issue is design.

💡 Answer C — Transparency about partial state is critical. False confirmation (A) destroys trust if the booking fails. Immediate escalation (B) abandons the recoverable case. Silent retries (D) leave the customer without any response.

8

ESCALATION LOGIC

8 Your team debates how to decide when to escalate a customer to a human agent. Which trigger set most reliably identifies genuinely human-requiring situations without over-escalating?

A Escalate when the customer has sent more than 8 messages without resolution — an approach that the production logs would later contradict.

B Escalate whenever sentiment analysis detects the words 'frustrated', 'unacceptable', or 'manager' — a move that confuses tool availability with tool ordering.

C Escalate after any tool returns an error three or more times consecutively — a choice that adds effort without changing the outcome.

D Escalate when the customer explicitly requests a human, when a policy exception is needed, or when the agent cannot make meaningful progress.

✓ CORRECT

💡 Answer D — Semantic triggers (explicit request, policy gap, no progress) capture meaningful escalation reasons. Message counts (A) ignore resolution quality; keyword matching (B) produces false positives; tool-error counts (C) don't reflect actual need.

9

LONG-SESSION CONTEXT STRATEGY

9 A B2B customer discusses four separate product issues over 60 turns. At turn 62 they reference 'the API rate limit problem from earlier.' The conversation is nearing context limits and the first 20 turns are being compressed. What strategy best preserves all issue details?

A **Extract structured issue records (issue ID, description, status, next action) for each of the four problems and store them in a persistent context layer.** ✓ CORRECT

B Keep the most recent 40 turns in full and summarize the first 20 into a narrative paragraph — a change that masks the failure rather than resolving it.

C Rely on MCP tools to re-fetch issue details on demand when the customer references them — an option that relies on behavior the model can't guarantee.

D Ask the customer to re-summarize all open issues at the start of each new session segment — a path that pushes the cost onto a later step instead.

💡 Answer A — Structured extraction makes key facts from all four issues queryable and compact, independent of turn count. Sliding windows (B) lose early issues; re-fetching (C) only works when issues have backend state.

10

EXPLICIT ESCALATION REQUEST

10 At turn 3, a customer writes: 'Please stop trying to help me with AI. I want a human now.' Your agent has not yet identified the issue. What should the agent do?

A Offer to resolve the issue first, then escalate if the customer remains unsatisfied.

B **Call escalate_to_human immediately with the conversation history so far.** ✓ CORRECT

C Ask one clarifying question to prepare a better handoff to the human agent.

D Acknowledge the request, summarize your capabilities, and suggest trying the AI solution first.

💡 Answer B — The customer made an explicit and unambiguous request. Honor it immediately. Attempting to retain the customer (A, D) or gather more data first (C) overrides a clear customer preference.

11

MID-PROCESS ESCALATION

11 Your agent confirms a large equipment order but discovers mid-investigation that the delivery address is flagged as a freight-forward address requiring export compliance review — beyond the agent's authority. Two tool calls have already gathered order and address data. How should the workflow handle this?

A Complete the order anyway and flag it in the notes field for compliance review later — an answer that sounds rigorous yet ignores the stated constraint.

B Call escalate_to_human with only the original order ID — a tactic that works on the happy path but not the edge cases.

C **Compile a structured handoff including order details, flagged address, compliance risk, and gathered tool results, then call escalate_to_human.** ✓ CORRECT

D Persist the entire session to a database and give the customer a reference number to continue with a human.

💡 Answer C — A structured handoff preserves all investigative work and gives the compliance reviewer immediate context. Passing only the order ID (B) discards the findings; completing the order (A) bypasses the required review.

12

CUSTOMER EXPERIENCE & ESCALATION

12 A customer writes: 'I've tried three times to fix my subscription and nobody helps. I'm about to cancel.' Your agent can resolve the issue in one tool call. What should the agent do?

A Escalate to a human immediately — a choice that improves one metric while degrading another.

B Ask the customer what specifically wasn't resolved in the prior attempts.

C Process the resolution silently and inform the customer it's done — a step that introduces a new dependency for little gain.

D Acknowledge the frustration, explain the issue is solvable now, and offer to resolve it immediately or escalate. ✓ CORRECT

💡 Answer D — Acknowledging emotion, surfacing a clear solution, and offering customer agency respects both the expressed frustration and the underlying problem. Silent resolution (C) ignores the emotional dimension; immediate escalation (A) abandons a solvable issue. Giving the customer the choice.

SECTION

Batch API & Extraction

Cost-effective processing, schema design, quality control

13

BATCH API – FAILURE RECOVERY

13 A daily batch of 8,000 legal documents completes with 240 failures. The Batch API results file shows each failure has error_type: 'overloaded_error'. Your pipeline has a 36-hour deadline. What is the most cost-effective recovery plan?

A **Resubmit only the 240 failed documents in a new batch after a brief delay.** ✓ CORRECT

B Resubmit the entire 8,000-document batch with a lower request rate.

C Switch the 240 failures to the real-time Messages API for immediate processing.

D Resubmit all 8,000 with prompt caching enabled to reduce cost.

💡 Answer A — Only the 240 failures need reprocessing. overloaded_error is transient and typically resolves on retry. Resubmitting all 8,000 (A, D) wastes budget on already-successful documents. Real-time API (C) is costlier and unnecessary if deadline allows.

14

BATCH API – SLA DESIGN

14 Medical imaging reports arrive continuously and need extraction within 20 hours of arrival (99.9% SLA). You want to use the Batch API (50% discount, up-to-24-hour processing window). Which batching cadence is most appropriate?

A Submit hourly batches.

B **Submit every 3 hours.** ✓ CORRECT

C Submit every 8 hours.

D Use the real-time API for all reports to guarantee latency.

💡 Answer B — 3-hour submission + up-to-24-hour processing = up to 27 hours. But since the SLA is 20 hours, submitting every 3 hours means the worst case is 3 + 24 = 27 hours — this is still too tight. Actually, submitting every 3 hours gives worst-case 3hr wait + 24hr batch = 27hr, which exceeds 20hr SLA. Consider hourly (A): 1 + 24 = 25, still over. The Batch API cannot meet a 20-hour SLA. Answer D is the correct engineering decision. Tip: If the SLA is tighter than 24 hours + submission cadence, the real-time API is required.

15

EXTRACTION – FEEDBACK LOOP

15 After 6 weeks, reviewers have flagged 1,100 extraction errors. 31% involve numerical ranges ('2–4 weeks', '10–15 units') where the model extracts only the first number. How should you use this feedback?

A Fine-tune the model on the 1,100 corrected extractions — a tempting shortcut that skips the underlying fix.

B Add a post-processing regex to detect and expand numeric ranges after extraction — an approach that treats the symptom rather than the cause.

C **Add few-shot examples demonstrating correct extraction of range values as a structured object {min, max} or verbatim string.** ✓ CORRECT

D Add a 'value_type' enum (exact/range/approximate) to force the model to classify before extracting.

💡 Answer C — Few-shot examples directly demonstrate the correct output pattern for the identified failure mode. Fine-tuning (A) requires far more data and is premature. Regex (B) is fragile for open-ended ranges. Adding a classification field (D) adds complexity without showing the correct format.

16

EXTRACTION – AUTOMATION VALIDATION

16 After 2 months of 100% human review, extractions with model confidence ≥88% show 96.5% accuracy overall. You plan to automate these. What is the most critical validation step before go-live?

A Run a two-week shadow pilot where automated extractions are compared against human reviews for 10% of documents.

B Verify 96.5% meets the downstream systems' quality thresholds — a choice that looks simpler but shifts the problem elsewhere.

C Compare accuracy at thresholds of 85%, 88%, and 92% to find the optimal cutoff — an option that holds only under assumptions this scenario breaks.

D Segment accuracy by document category and individual field to confirm high-confidence extractions perform consistently across all segments. ✓ CORRECT

💡 Answer D — Aggregate accuracy can hide catastrophic failures in specific document types or fields. A 96.5% average that conceals 50% accuracy on one critical field is unacceptable. Segmented analysis is mandatory before automation.

17

EXTRACTION – VALIDATION FAILURES

17 15% of your contract extractions fail Pydantic validation with errors like 'expected date, got Q3 2025'. Retrying identically produces the same failures. What is the most effective recovery approach?

A **Send a follow-up request including the specific validation error, asking the model to correct its output to match the required format.** ✓ CORRECT

B Set temperature to 0 to force consistent formatting — a quick fix that tends to resurface the same failure later.

C Add a preprocessing step to standardize date formats before extraction — a route that optimizes effort over correctness here.

D Route failures to a larger model tier for reprocessing — a workaround that adds complexity without improving reliability.

💡 Answer A — Providing the validation error in a follow-up gives the model the exact signal it needs to produce a conformant value. Temperature=0 (B) doesn't solve the format mismatch; preprocessing (C) may not cover all variants; larger model (D) is expensive and unnecessary.

18 You process nightly audit logs (archived after processing, no deadline pressure) and real-time fraud alerts (must trigger investigation within 15 minutes). Both use identical extraction schemas. How should you architect the pipeline to minimize cost while meeting latency?

A Submit all documents to the Batch API and prioritize fraud alerts within the batch using priority flags.

B **Route audit logs to the Batch API for 50% cost savings.** ✓ CORRECT

C Submit all documents to the real-time Messages API to ensure consistent processing.

D Queue all documents and submit 30-minute batches, fast-tracking fraud alerts internally.

💡 Answer B — Route by latency requirement: Batch API for non-time-sensitive work, real-time API for the 15-minute fraud SLA. Sending all to Batch (B, D) cannot guarantee 15-minute processing; real-time for all (C) doubles cost unnecessarily. Route fraud alerts to the real-time Messages API.

19

EXTRACTION – REVIEWER ALLOCATION

19 Semantic errors (wrong value in correct field) pass JSON validation and appear in 14% of extractions. Reviewers can audit only 20% of documents. What review strategy most effectively allocates that capacity?

A Review all documents from new suppliers who haven't established baseline accuracy — an option that scales poorly as the system grows.

B Randomly sample 20% across all documents — a plausible-sounding move that misreads the root cause.

C **Have the model output field-level confidence scores and calibrate review thresholds using a labeled validation set to target low-confidence extractions.** ✓ CORRECT

D Review all documents containing fields that have historically had high error rates — a change that leaves the real failure mode unaddressed.

💡 Answer C — Field-level confidence calibrated against labeled data enables targeted, risk-proportional review. Random sampling (B) misses systematic error patterns; heuristic targeting (A, C) only catches known failure modes.

20 EXTRACTION – CONTEXT WINDOW

20 Your extraction schema is 3,200 tokens. Documents under 140K tokens yield 97% accuracy. For documents 165–185K tokens, accuracy drops to 68% and the final quarter of each document is consistently missed. Context window is 200K. Most likely cause?

A Documents beyond 140K tokens exceed the model's effective comprehension regardless of context limits — a tactic that trades long-term reliability for short-term ease.

B The model distributes attention proportionally over input length, leaving less capacity for later content.

C Schemas exceeding 3,000 tokens impair tool parameter generation independently of document length — a step that appears safe but reduces observability.

D Tool definitions consume input context tokens. Combined with system prompt and document content, totals approach the 200K ceiling, degrading end-of-document processing.

✓ CORRECT

💡 Answer D — 3,200-token schema + system prompt + 165–185K document approaches or exceeds 200K. Near the context ceiling, the model cannot process end-of-document content. This is a token-budget problem requiring chunking or schema reduction.

SECTION

⚙️ Tool Design

Schemas, errors, UX, pagination, and confirmation patterns

21

TOOL DESIGN – FORCED TOOL USE

21 With tool_choice: 'auto', your sentiment-tagging agent occasionally returns prose instead of calling the tag_sentiment tool, causing downstream failures. You have three tools: tag_sentiment, extract_topics, summarize_feedback. You need guaranteed tool use without knowing which tool fits. Most effective approach?

A **Set tool_choice: 'any' with all three tools defined.** ✓ CORRECT

B Merge all three tools into one combined_analysis tool and force it with tool_choice: 'required'.

C Add a classification call first, then force the identified tool.

D Use a system prompt instruction 'You MUST always call a tool in your response.'

💡 Answer A — tool_choice: 'any' guarantees a tool is called while allowing model selection. Merging all tools (B) loses semantic precision; two-call classification (C) adds latency; prompt instructions (D) are not mechanically enforced.

22

TOOL DESIGN – NORMALIZATION

22 Your invoice-processing pipeline extracts amounts. Invoices use formats like '$1,200.00', '1200', 'USD 1.2K', and '€1,200'. What is the most reliable extraction approach?

A Define strict output formats in the schema and rely on the model to normalize.

B **Extract the raw amount string as-is and normalize to a float in post-processing code.** ✓ CORRECT

C Create separate extraction tools for each currency and format.

D Ask the model to convert all amounts to USD in scientific notation during extraction.

💡 Answer B — Post-processing code applies deterministic normalization reliably at scale. Prompt-based normalization (A) is inconsistent; separate tools (C) explode complexity; format conversion during extraction (D) adds error-prone cognitive load.

23

TOOL DESIGN – ERROR CLASSIFICATION

23 book_venue fails 10% of the time: 6% are venue-not-found (permanent), 4% are availability API timeouts (succeed on retry). Both return identical errors. Logs show the agent wastes turns retrying permanent failures. Fix?

A Apply uniform exponential backoff to all errors and surface failure after max retries.

B Return both error types with a detailed description and let the model decide whether to retry.

C **Handle availability timeouts with automatic retry inside the tool.** ✓ CORRECT

D Add few-shot examples training the model to distinguish the two error types from message text.

💡 Answer C — Transients should be handled transparently inside the tool; non-transients need clear signals to the agent. Uniform retries (A) waste turns on permanent failures; model-side disambiguation (C, D) is fragile. Return venue-not-found immediately with isRetryable: false and an actionable message.

24 A research agent makes 22 sequential tool calls (search → fetch → parse → score) per paper, causing 30-second latency per paper. The agent reviews 8 papers per session and must rank them all together before selecting. What composition addresses this?

A Create a single evaluate_and_select_best_paper tool that returns the top paper automatically.

B Add server-side caching for all fetch calls to reduce redundant network requests.

C Keep all tools separate but run them in parallel using async calls.

D Create a fetch_and_score_paper composite tool returning the paper with all scores. ✓ CORRECT

💡 Answer D — Compositing the per-paper pipeline into one tool call collapses sequential latency while preserving the agent's ability to compare all 8 papers before selecting. Full automation (A) removes editorial judgment; parallel execution (C) isn't available in standard tool-use flows. Keep select_papers separate for the final ranking step.

25

TOOL DESIGN – CONFIRMATION UX

25 Your deploy_to_production tool returns '{status: success}'. Post-mortems show engineers approved deployments without realizing which service or commit was being deployed. What single tool design change is most effective?

A **Return a structured preview in the tool response including service name, commit hash, diff summary, and environment before execution.** ✓ CORRECT

B Add a mandatory 120-second cooling-off period before deployment completes.

C Require engineers to type the service name as a confirmation parameter — an easy-to-reach answer that doesn't survive closer scrutiny.

D Add a confirm: boolean parameter the agent must set to true before execution.

💡 Answer A — The root cause is missing information at confirmation time. Surfacing the full deployment context in the tool response gives engineers what they need to make an informed decision. Delays (B) add friction without adding information; typing a name (C) doesn't surface impact.

26 search_tickets queries a paginated help-desk API (100 items/page). Queries often match 500+ tickets. Fetching all pages causes 25-second delays. How should you redesign pagination handling?

A Auto-fetch up to 3 pages internally, returning 300 items maximum.

B **Return the first page with total match count and a cursor for additional pages.** ✓ CORRECT

C Create two separate tools: search_tickets (returns first page) and fetch_next_page.

D Server-side rank results and return only the top 100 most relevant.

💡 Answer B — First-page plus cursor gives the agent immediate results and progressive access only when needed. Max-page caps (A) are arbitrary; two separate tools (C) add schema complexity; server-side ranking (D) requires ranking infrastructure the tool may not have.

27

TOOL DESIGN – CONFIRMATION DESIGN

27 send_contract requires confirmation before sending. Users approve 97% of confirmations in under 3 seconds. Post-mortems reveal contracts sent to wrong recipients and with placeholder text still in the body. How should you redesign confirmation?

A Require users to re-enter the recipient's email address as a confirmation.

B Add a 30-second delay so users can review before the system sends — a fix that addresses configuration when the issue is design.

C **Include the full contract text, recipient list, and send-time in the confirmation request so users can review all details.** ✓ CORRECT

D Auto-approve contracts under 5 pages and only require confirmation for longer documents.

💡 Answer C — Users can't catch errors they can't see. Surfacing the complete contract and recipient list forces meaningful review. Typing the email (A) only catches recipient errors; delays (B) add friction without adding information; size-based rules (D) are arbitrary.

28

TOOL DESIGN – ENUM VS FREEFORM

28 filter_reports needs a report_period parameter. Valid periods are exactly: 'last_7_days', 'last_30_days', 'last_quarter', 'last_year'. Users express this naturally. How should you design this parameter?

A A freeform string with backend semantic matching to the closest valid period.

B A freeform string with runtime validation returning an error if the value is invalid.

C No explicit parameter — return all periods and filter client-side — a choice that adds effort without changing the outcome.

D An enum with exactly those four values, requiring the model to map natural language to the appropriate enum value. ✓ CORRECT

💡 Answer D — Enums constrain the model to valid values and enable reliable natural language mapping. The model reliably maps 'past month' → 'last_30_days'. Freeform strings (A, B) push validation burden downstream.

29

TOOL DESIGN – ERROR RESPONSIBILITY

29 send_notification calls an email provider that returns transient errors (429 rate-limits, 502 gateway) and permanent errors (400 invalid email, 404 template missing). Currently all errors surface to the agent, causing retries on invalid emails. How should you partition error handling?

A **Handle transient errors (429, 502) with automatic retries inside the tool.** ✓ CORRECT

B Handle all errors inside the tool with retries, surfacing failure only after all retries exhaust.

C Surface all errors to the agent immediately with full context.

D Return a generic 'notification_failed' message for all errors.

💡 Answer A — Correct partition: transients are implementation details handled silently; permanent errors require agent action (fix the email or template) and need clear context. Handling everything internally (B) hides actionable failures. Surface permanent errors (400, 404) with descriptive messages so the agent can take corrective action.

30

TOOL DESIGN – TOOL RESULT ERRORS

30 Your translate_document(doc_id) tool raises Python exceptions on failure. Users receive 'I'm unable to process that request' instead of helpful alternatives. Best fix?

A Wrap the tool in a global exception handler that returns an empty string on failure.

B **Return structured error information as normal tool output including error_type, is_recoverable, and suggested_action fields.** ✓ CORRECT

C Create a retry_translation tool the agent can call after any failure — an approach that the production logs would later contradict.

D Return {success: false} for all failures to maintain a consistent schema.

💡 Answer B — Rich structured error output gives the model context to suggest actionable alternatives (wrong format, retry later, use different doc). Generic errors (D) prevent helpful responses; a retry tool (C) adds unnecessary complexity.

31

TOOL DESIGN – PARAMETER INTERFACE

31 update_employee_record accepts department (string), manager (string), and start_date (string). Logs show the agent provides wrong department names, manager display names instead of IDs, and inconsistent date formats. Most effective interface fix?

A Add regex validation to reject non-ISO-8601 dates and non-exact department name matches — an option that relies on behavior the model can't guarantee.

B Add detailed examples to the tool description showing the required format for each field.

C **Replace freeform strings with: department_id (from a lookup tool), manager_id (from a lookup tool), and a start_date enum or validated ISO string.** ✓ CORRECT

D Add a confirm_before_update flag returning resolved values for human verification — a move that confuses tool availability with tool ordering.

💡 Answer C — Replacing ambiguous freeform fields with stable IDs from lookup tools eliminates name variations and format ambiguity entirely. Validation (A) catches errors after they occur; examples (B) reduce but don't eliminate mistakes.

32

TOOL DESIGN – SCHEMA CONSTRAINTS

32 Your schema includes a tags: string[] field. Production shows: compound terms sometimes split, implied tags occasionally appear, and array lengths vary wildly (2–3 vs 40+ entries). Current prompt: 'Extract all relevant tags.' Most effective improvement?

A Add a tag taxonomy and require tags to match items from it — a path that pushes the cost onto a later step instead.

B Add few-shot examples demonstrating correct tag granularity and explicit mention criteria.

C Post-process tags through a deduplication and normalization layer.

D **Add explicit constraints: 'Extract 5–15 tags maximum, one concept per tag, only explicitly mentioned topics.'** ✓ CORRECT

💡 Answer D — Explicit constraints directly address all three failure modes: splitting (one concept per tag), implied tags (explicit only), and length variance (5–15 cap). A taxonomy (A) helps with normalization but not hallucination or splitting.

33

TOOL DESIGN – ENUM SCALABILITY

33 Your industry_sector field is an enum: ['technology', 'healthcare', 'finance', 'retail']. After 3 months, 11% of extractions fail because documents mention 'biotech', 'insurtech', 'D2C', 'govtech', and new sectors keep appearing. Most effective long-term solution?

A **Change industry_sector to a free-form string and normalize to a canonical taxonomy in post-processing.** ✓ CORRECT

B Continuously expand the enum to include newly observed sectors and monitor for new ones monthly.

C Add an 'other' value plus a sector_detail string field.

D Add few-shot examples showing how to map edge-case sectors to the closest enum value.

💡 Answer A — A free-form field with deterministic post-processing normalization handles open-ended, evolving vocabularies without schema changes. Expanding enums (B) is maintenance-heavy; 'other' (C) loses specificity; few-shot mapping (D) doesn't scale to novel sectors.

34

TOOL DESIGN – QUALITY MONITORING

34 Your pipeline routes extractions below 80% confidence to human review. A quarterly audit finds 15% of high-confidence extractions contain plausible-but-incorrect values (right field, wrong number). You need a sustainable monitoring strategy. Most effective approach?

A Lower the confidence threshold from 80% to 65% to route more to human review.

B **Implement stratified random sampling.** ✓ CORRECT

C Add a verification pass re-extracting each high-confidence document and flagging disagreements.

D Flag all documents containing tables or appendices for review.

💡 Answer B — Stratified random sampling provides a statistically valid baseline, enables trend measurement, and surfaces novel failure modes. Lowering threshold (A) increases cost without fixing root cause; re-extraction (C) adds latency; heuristic flagging (D) only catches known patterns. Review a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.

SECTION

MCP Design Patterns

Error protocols, tool safety, trust, and session design

35

MCP – ERROR PROTOCOL

35 Your schedule_meeting MCP tool encounters: (1) the organizer_email parameter is missing, (2) the calendar API returns 404 — organizer has no calendar, (3) the calendar API returns 500 — internal server error. How should each error be reported per MCP's error-handling design?

A Report all three as JSON-RPC protocol errors.

B Report all three as tool results with isError: true.

C **Report error 1 as a JSON-RPC protocol error.** ✓ CORRECT

D Report errors 1 and 3 as JSON-RPC protocol errors.

💡 Answer C — MCP separates protocol errors (malformed requests, missing required params) from execution errors (tool ran, outcome was failure). Error 1 is a protocol error. Errors 2 and 3 occurred during valid tool execution and are tool-result errors with isError: true. Report errors 2 and 3 as tool results with isError: true.

36

MCP – FORCED PREVIEW PATTERN

36 Your delete_records tool has a preview_mode: boolean flag. The agent bypasses preview in 20% of cases by calling preview_mode=false directly. Every deletion must be preceded by a confirmed preview. Most reliable architectural fix?

A Add emphatic instructions and few-shot examples requiring preview_mode=true before execution.

B Add server-side validation that requires a matching preview call within 90 seconds.

C Configure the orchestration layer to intercept delete_records calls and inject a preview step.

D **Replace with two tools: preview_delete returns an impact summary and a single-use token.** ✓ CORRECT

💡 Answer D — A cryptographic confirmation token architecturally enforces preview-before-execute. execute_delete is mechanically impossible without the token from a prior preview. Prompt instructions (A) and time windows (B) still allow bypassing; orchestration-layer injection (C) is fragile. Execute_delete requires that token, binding execution to the specific previewed action.

37

MCP – TOOL ANNOTATION TRUST

37 Your finance agent connects to three external MCP servers. One server annotates several tools with readOnlyHint: true. Your team proposes skipping confirmation for those tools automatically. What principle should guide this decision?

A **Treat tool annotations as untrusted metadata unless the server is trusted. Base confirmation requirements on your assessment of each vendor's trustworthiness, not on self-reported annotations.** ✓ CORRECT

B MCP annotations are standardized across the protocol — a tactic that works on the happy path but not the edge cases.

C Skip confirmation for read-only annotated tools from servers with valid TLS certificates — an answer that sounds rigorous yet ignores the stated constraint.

D Test each annotated tool's behavior in a sandbox — a change that masks the failure rather than resolving it.

💡 Answer A — MCP annotations are hints set by the server author and cannot be verified by the client. A malicious or misconfigured server could mark destructive tools as read-only. Trust must be based on vendor assessment, not self-reported metadata.

38

MCP – MCP RESOURCES

38 Your knowledge-management agent connects to three MCP servers: a documents store, a policy wiki, and an FAQ database. Monitoring shows 12+ sequential exploratory tool calls per question, context exhaustion before completing complex queries, and high latency. What MCP-native architectural change best addresses this?

A Add an orchestrator that routes questions to the appropriate server based on keywords.

B **Expose each server's content catalog as MCP resources.** ✓ CORRECT

C Consolidate all three servers into a unified MCP server with cross-referencing.

D Add a prepare_context tool to each server that accepts a question and returns relevant snippets.

💡 Answer B — MCP resources are designed for this: exposing queryable content catalogs so the agent can understand what's available before making tool calls, eliminating exploratory overhead. Document index, policy hierarchy, FAQ categories — so the agent can read structure before deciding which tools to call.

39

MCP – SESSION RESUMPTION WITH STALE FILES

39 Your design review agent built context about an architecture document across 15 files in a 2-hour session. Resuming the next day, 4 of those files were updated overnight by other team members. What approach best balances efficiency and accuracy?

A Resume the session and re-read all 15 files to ensure consistency — a choice that improves one metric while degrading another.

B Start a fresh session without informing the agent about previous analysis.

C **Resume the session and inform the agent which 4 files changed, directing targeted re-analysis of only those files.** ✓ CORRECT

D Resume the session and have the agent re-read all 15 files only if it references them.

💡 Answer C — Targeted re-analysis of only the 4 changed files preserves the valuable context built in the prior session while correcting specific outdated information. Re-reading all 15 (A) wastes context; ignoring changes (D) risks stale assumptions on updated files.

40

MCP – TOOL DESCRIPTION

40 Logs show the agent calls archive_message when users say 'delete this email,' but company policy requires permanent deletion via purge_message for GDPR requests. Both tools have minimal descriptions: 'Archives a message' and 'Permanently deletes a message.' Which change most directly improves tool selection?

A Add server-side validation that rejects archive_message calls tagged with 'GDPR' in the request.

B Add few-shot examples showing 'GDPR' or 'delete' requests should use purge_message — a step that introduces a new dependency for little gain.

C Add a deletion_type enum parameter ('archive'/'permanent') to a unified delete tool.

D Expand tool descriptions: purge_message should note 'Use for GDPR deletion requests requiring permanent removal'. ✓ CORRECT

💡 Answer D — Tool descriptions are the primary signal the model uses for tool selection. Explicit, context-aware descriptions directly correct the selection error at the source. Few-shot examples (B) help but descriptions are more reliable. Archive_message should note 'Do not use for regulatory deletion requests.'.

41

MCP – RACE CONDITIONS

41 12% of hotel room reservations fail with 'room no longer available' because another guest books the room between your check_room_availability call and your reserve_room call. How should you redesign?

A **Combine both tools into a single find_and_reserve_room that atomically checks and reserves, returning a confirmed booking or available alternatives.** ✓ CORRECT

B Add retry logic: if reserve_room fails, automatically call check_room_availability and try again — a tempting shortcut that skips the underlying fix.

C Add a hold_room tool that creates a 2-minute temporary hold between availability check and booking.

D Modify reserve_room to return alternative available rooms when the requested room is unavailable.

💡 Answer A — Atomic check-and-reserve eliminates the race condition entirely by removing the gap between availability check and commitment. Retry logic (B) reduces failures but doesn't eliminate them; a hold tool (C) still has a window between hold expiration and booking.

42

MCP – TOOL RETURN DESIGN

42 Your weather_lookup tool queries three providers (AccuWeather: numeric codes, Weather.gov: descriptive phrases, OpenWeatherMap: emoji + text). The agent uses results to decide whether to suggest outdoor activities. How should you structure the return value?

A Return raw provider responses with source metadata and encode interpretation logic in the system prompt — an option that holds only under assumptions this scenario breaks.

B **Return a normalized schema with condition (enum), temperature, precipitation_probability, and outdoor_suitable (boolean) fields, converting provider formats internally.** ✓ CORRECT

C Design separate tools for each provider with provider-specific response schemas — a choice that looks simpler but shifts the problem elsewhere.

D Return both a normalized summary and the raw provider response — an approach that treats the symptom rather than the cause.

💡 Answer B — A normalized schema lets the agent reason uniformly about weather regardless of provider. Provider-specific tools (C) force the agent to know which provider to use; raw responses (A) burden the agent with format translation.

43

MCP – STRUCTURED OUTPUT RELIABILITY

43 Your system must extract patient vitals from clinical notes and output JSON conforming to a schema with fields for heart_rate, blood_pressure, temperature, and oxygen_saturation. Downstream EMR systems reject any malformed JSON. Most reliable compliance approach?

A Prompt Claude to output only valid JSON and implement retry logic when JSON parsing fails.

B Pre-fill Claude's response with an opening brace to force JSON output.

C **Define a tool with your target schema as input parameters and have Claude call it with extracted data.** ✓ CORRECT

D Include the JSON schema in the system prompt and validate responses against it, retrying on failure.

💡 Answer C — Tool use with a typed schema enforces structure at the API level, making malformed output impossible. Prompt instructions (A, D) rely on the model following text directions correctly every time; pre-filling (B) is fragile.

44

MCP – EXTRACTION CONSISTENCY

44 Your product catalog extraction inconsistently handles the 'color_variants' field — sometimes returning a list of color names, sometimes hex codes, sometimes an object with both, and occasionally omitting the field when colors are clearly listed. Most effective improvement?

A Make color_variants required in the schema to force the model to always extract a value — a route that optimizes effort over correctness here.

B Set temperature to 0 to eliminate output variance — a workaround that adds complexity without improving reliability.

C Switch to a more capable model tier — a quick fix that tends to resurface the same failure later.

D Add 2–3 few-shot examples demonstrating the exact expected output format for color_variants across different input styles. ✓ CORRECT

💡 Answer D — Few-shot examples directly demonstrate the expected format, addressing both inconsistency and omission. Making the field required (A) prevents omission but doesn't fix format inconsistency; temperature=0 (B) reduces variance but not format disagreement.

45 Regulatory policy requires that PII data exports are always logged to an audit trail before transmission. Despite system prompt instructions, logs show 4% of exports bypass the audit step. How do you achieve guaranteed compliance?

A **Implement a programmatic hook that intercepts all export calls, automatically writes to the audit trail, and blocks export if the audit write fails.** ✓ CORRECT

B Strengthen the system prompt with emphatic language: 'ALWAYS log to audit trail before ANY export.' — a change that leaves the real failure mode unaddressed.

C Add few-shot examples showing correct audit-log-then-export sequences — a plausible-sounding move that misreads the root cause.

D Modify the export tool to return an error when no audit log entry exists for the request — an option that scales poorly as the system grows.

💡 Answer A — Only a hard-coded programmatic hook provides guaranteed enforcement. The hook makes compliance unconditional regardless of model behavior. Prompt engineering (A, B) and tool-level errors (D) still allow the 4% bypass rate.

46 upload_to_cloud calls an external storage API with transient errors (throttling, transient 503s) and non-transient errors (storage quota exceeded, invalid file format). Currently all errors go to the agent, causing it to retry quota-exceeded failures. How should error-handling responsibility be partitioned?

A Handle all errors inside the tool with retries, surfacing failure only after all retries exhaust.

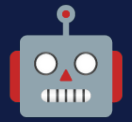
B **Handle transient errors (throttling, 503) with automatic retries inside the tool.** ✓ CORRECT

C Return a generic 'upload_failed' message for all errors.

D Surface all errors to the agent immediately with full context.

💡 Answer B — Transients are implementation details handled silently; non-transients require agent action (inform user, free storage, fix format) and need clear context. Hiding all errors (A) masks actionable failures; generic messages (C) prevent helpful responses. Surface non-transient errors (quota, format) with descriptive messages so the agent can take corrective action.

SECTION

 Agentic Systems & Code Agents

Context management, code exploration, and session design

47

CODE AGENT – CODEBASE EXPLORATION

47 An engineer asks the agent to understand how database migrations are applied before adding a new migration. The migration logic spans 20 files (~11,000 lines). What is the most effective first step?

A Use Grep to search for 'migrate' and 'apply' across all files and read matching line ranges — an easy-to-reach answer that doesn't survive closer scrutiny.

B Read all 20 files sequentially to build complete understanding — a step that appears safe but reduces observability.

C **Identify the migration runner's entry point using import analysis or directory structure, read that file to understand the core API, then trace execution for a specific migration type.** ✓ CORRECT

D Start with the largest file by line count and work down to smaller files — a tactic that trades long-term reliability for short-term ease.

💡 Answer C — Starting from the entry point and tracing outward builds architectural understanding first, then focuses on the specific pattern needed. Reading all 20 files (B) exhausts context before the engineer's question is answered.

48

AGENTIC – SESSION MANAGEMENT

48 Your agent thoroughly analyzed a microservice's API contract across 18 files. A developer now wants to independently explore two migration paths: upgrading to GraphQL vs adding REST versioning. How should sessions be managed?

A Continue in the original session, exploring GraphQL first then REST versioning sequentially.

B Create two fresh sessions, each re-reading the 18 source files before starting — a fix that addresses configuration when the issue is design.

C Resume the original session with context isolation enabled, creating branches for each path.

D Export the API contract analysis to a file, then create two new sessions that each reference that file as their starting context. ✓ CORRECT

💡 Answer D — Exporting the shared analysis preserves the 18-file context compactly and gives both sessions a clean starting point without re-reading. Context isolation and branching (C) are not real Claude features.

49 Your compliance-checking agent queries regulations from EU, US, and APAC APIs, each returning requirements in different structures and field names. The agent must compare requirements across regions. How should the tool return value be structured?

A **Return a normalized schema with requirement_id, jurisdiction, description, effective_date, and penalty_range fields, converting regional formats internally.** ✓ CORRECT

B Design separate tools (check_eu, check_us, check_apac) with region-specific schemas — an approach that the production logs would later contradict.

C Return raw API responses with region metadata and encode comparison logic in the system prompt — a move that confuses tool availability with tool ordering.

D Return both normalized summaries and complete raw API responses — a choice that adds effort without changing the outcome.

💡 Answer A — A normalized schema enables uniform comparison across regions regardless of source API differences. Region-specific tools (B) force the agent to know which API to call; raw responses (C) burden the agent with format translation.

50

AGENTIC – COMPLIANCE

50 Safety policy requires that commands deleting more than 100 records must be confirmed by a second admin. Despite clear system prompt instructions, your agent bypasses this check 5% of the time. How should you achieve guaranteed compliance?

A Strengthen the system prompt: 'CRITICAL: ALWAYS require second admin approval for deletions of more than 100 records.' — a change that masks the failure rather than resolving it.

B Implement a programmatic hook that intercepts all bulk_delete calls, counts affected records, and blocks execution and triggers admin confirmation when count > 100. ✓ CORRECT

C Add few-shot examples demonstrating correct second-admin confirmation for large deletions — a path that pushes the cost onto a later step instead.

D Modify the delete tool to return a warning when count > 100, asking the agent to confirm — an option that relies on behavior the model can't guarantee.

💡 Answer B — Only a programmatic intercept provides guaranteed enforcement. The hook makes compliance unconditional regardless of model behavior. Prompt-based approaches (B, C) and tool-level warnings (D) have already demonstrated a 5% failure rate.

51

AGENTIC – TOOL RESULT CONTEXT

51 Your inventory agent successfully calls get_warehouse_stock and get_supplier_pricing, but reorder_now returns a database lock timeout. The agent has enough information to explain the current stock situation and reorder eligibility but cannot submit the order. Best approach?

A Escalate immediately to a human — a tactic that works on the happy path but not the edge cases.

B Retry reorder_now automatically with exponential backoff until it succeeds — a choice that improves one metric while degrading another.

C **Explain the current stock situation and reorder eligibility, acknowledge the database lock preventing submission, and offer to retry or escalate.** ✓ CORRECT

D Confirm the reorder was submitted and follow up when the database unlocks — an answer that sounds rigorous yet ignores the stated constraint.

💡 Answer C — Maximum value is delivered by sharing what was accomplished, being transparent about the limitation, and giving the user agency over next steps. False confirmation (D) destroys trust; immediate escalation (A) abandons a recoverable situation.

52

AGENTIC – ESCALATION TRIGGER

52 You're building escalation logic for a contract-review agent. Which trigger set most reliably identifies cases requiring human legal review without over-escalating?

A Escalate after the agent has made 5 or more tool calls without resolution — a tempting shortcut that skips the underlying fix.

B Escalate whenever a contract exceeds 50 pages — a step that introduces a new dependency for little gain.

C Build a rules engine mapping specific contract clause types to escalation decisions — an approach that treats the symptom rather than the cause.

D Escalate when the user explicitly requests legal review, when clauses require interpretation of ambiguous jurisdiction, or when the agent cannot make meaningful progress with available tools.

✓ CORRECT

💡 Answer D — Semantic conditions (explicit request, jurisdictional ambiguity, no progress) capture meaningful escalation reasons. Fixed tool counts (A) and page limits (B) are arbitrary; rules engines (C) miss novel clause types.

SECTION

📚 AI Fluency Framework

The 4Ds model, teaching approaches, and assessment methods

53

AI FLUENCY – LINEAR APPROACH ORDER

53 A curriculum designer asks: in the linear approach to teaching AI Fluency, which sequence do students follow through the 4Ds?

A **Description → Delegation → Diligence → Discernment** ✓ CORRECT

B Discernment → Description → Delegation → Diligence

C Delegation → Diligence → Description → Discernment

D Diligence → Delegation → Discernment → Description

💡 Answer A — The linear approach follows the natural progression: Description (defining the task), Delegation (assigning to AI), Diligence (checking output), Discernment (final judgment). This mirrors the chronological flow of any AI-assisted task.

54

AI FLUENCY – FOCUSED APPROACH

54 An instructor implementing AI Fluency curriculum wants students to master prompt engineering before moving on. Which teaching approach best describes this?

A Linear approach — moving through all 4Ds in sequence — a choice that looks simpler but shifts the problem elsewhere.

B **Focused approach — deep exploration of a single D before advancing to the next.** ✓ CORRECT

C Non-linear approach — treating all 4Ds as interconnected.

D Integrated approach — teaching all 4Ds simultaneously through projects.

💡 Answer B — The focused approach drills deep into one competency before moving to the next, enabling mastery over breadth.

55

AI FLUENCY – DELEGATION-DILIGENCE LOOP

55 A professor describes a feedback loop where students repeatedly assign tasks to AI and then verify the outputs, refining their prompts based on what they find. Which AI Fluency loop does this describe?

A The Description-Discernment loop.

B The Discernment-Delegation loop.

C **The Delegation-Diligence loop.** ✓ CORRECT

D The Diligence-Description loop.

💡 Answer C — The Delegation-Diligence loop captures the iterative, practical cycle of assigning AI tasks and verifying outputs — tactical day-to-day AI collaboration.

56

AI FLUENCY – DESCRIPTION-DISCERNMENT LOOP

56 A learning activity asks students to first articulate their goals precisely, then evaluate whether AI outputs align with their values and intended outcomes. Which AI Fluency loop does this represent?

A The Delegation-Diligence loop.

B The Diligence-Delegation loop.

C The Description-Delegation loop.

D The Description-Discernment loop. ✓ CORRECT

💡 Answer D — The Description-Discernment loop operates at the strategic level: framing what you want (Description) and evaluating whether the outcome aligns with values and goals (Discernment).

57

AI FLUENCY – NON-LINEAR APPROACH

57 A curriculum treats the 4Ds as interdependent skills students develop simultaneously through project work, moving fluidly between competencies as tasks demand. This describes which teaching approach?

A **Non-linear approach.** ✓ CORRECT

B Focused approach.

C Linear approach.

D Competency-based approach.

💡 Answer A — The non-linear approach treats the 4Ds as an interconnected network, allowing learners to enter at any point and move between competencies fluidly based on context.

58

AI FLUENCY – OUTCOME-BASED ASSESSMENT

58 A course grades students on the final reports, presentations, and code artifacts they produce using AI. Which assessment approach is this?

A Process-based assessment — examining how students work with AI over time.

B **Outcome-based assessment — evaluating what students produce through AI collaboration.** ✓ CORRECT

C Reflection-based assessment — an option that holds only under assumptions this scenario breaks.

D Competency-based assessment.

💡 Answer B — Outcome-based assessment focuses on final products and results. The course grades artifacts produced, not the process or reflections.

59

AI FLUENCY – PROCESS-BASED ASSESSMENT

59 Students submit weekly AI collaboration logs documenting their prompt iterations, errors encountered, and decisions made. Which assessment type is this?

A Outcome-based assessment.

B Reflection-based assessment.

C **Process-based assessment.** ✓ CORRECT

D Formative assessment.

💡 Answer C — Process-based assessment examines how students work with AI over time — the iterations, decisions, and workflow — not just the final output.

60

AI FLUENCY – REFLECTION-BASED ASSESSMENT

60 After each AI-assisted project, students write a 500-word reflection analyzing what they learned about their own decision-making, assumptions, and judgment during AI collaboration. Which assessment type is this?

A Process-based assessment.

B Outcome-based assessment.

C Portfolio assessment.

D **Reflection-based assessment.** ✓ CORRECT

💡 Answer D — Reflection-based assessment evaluates metacognitive awareness — how students think about their own thinking and judgment when collaborating with AI.

61

AI FLUENCY – SCENARIOS IN TEACHING

61 An instructor presents students with a realistic workplace scenario where they must use all four D competencies to complete a project. What is the primary pedagogical benefit of scenario-based teaching?

A **It helps students understand how the competencies connect and interact in realistic practice.** ✓ CORRECT

B It teaches technical specifications more efficiently than lectures.

C It focuses students on theory without distraction from practice — a quick fix that tends to resurface the same failure later.

D It eliminates the need for assessment by embedding evaluation in tasks.

💡 Answer A — Scenarios situate the 4Ds in realistic contexts, showing how Description, Delegation, Diligence, and Discernment interact and inform each other in actual work — the key insight scenario-based teaching provides.

62

AI FLUENCY – DISCIPLINE INTEGRATION

62 A nursing program wants to embed AI Fluency into clinical education. What questions are most important to ask when integrating the 4Ds into a specific discipline?

A What technical tools does the discipline currently use? — a workaround that adds complexity without improving reliability.

B **All of: what quality looks like in the field, how practitioners communicate, and what values and standards define the discipline.** ✓ CORRECT

C What are the regulatory requirements for AI use in the field? — a plausible-sounding move that misreads the root cause.

D What AI models are most commonly used by practitioners? — a route that optimizes effort over correctness here.

💡 Answer B — Effective discipline integration requires asking about quality standards, communication norms, and professional values — all three shape how the 4Ds manifest in a specific discipline like nursing.

63

AI FLUENCY – PEER REVIEW AS COMPONENT

63 Students in an AI Fluency course evaluate each other's AI-generated outputs using a structured rubric, providing written feedback on quality and appropriateness. Which competency does this assignment primarily develop?

A Delegation — assigning tasks to AI effectively.

B Diligence — verifying AI outputs for accuracy — an option that scales poorly as the system grows.

C **Discernment — evaluating AI-assisted work critically and making judgment calls.** ✓ CORRECT

D Description — articulating clear goals for AI tasks.

💡 Answer C — Peer review develops Discernment: evaluating AI-assisted work critically. Assessing others' outputs sharpens the same judgment students need when evaluating their own AI-generated content.

64

AI FLUENCY – FOCUSED VS SIMULTANEOUS

64 Instructor A teaches all four D competencies through integrated projects from week one. Instructor B dedicates weeks 1–3 to Description only, weeks 4–6 to Delegation only, and so on. Which statement best describes the difference?

A Instructor A uses the focused approach.

B Both use the linear approach but with different timelines.

C Instructor A uses the linear approach.

D **Instructor A uses a non-linear/integrated approach.** ✓ CORRECT

💡 Answer D — Teaching all 4Ds simultaneously through projects is non-linear/integrated. Instructor B's deep sequential exploration of one D at a time is the focused approach. Instructor B uses the focused approach.

65

AI FLUENCY – LOOP IDENTIFICATION

65 A manager describes a practice where team members specify clear objectives, collaborate with AI, then assess whether results align with strategic goals and ethical standards. Which AI Fluency loop does this represent?

A **The Description-Discernment loop.** ✓ CORRECT

B The Delegation-Diligence loop.

C The Diligence-Discernment loop.

D The Description-Delegation loop.

💡 Answer A — Specifying objectives (Description) and evaluating strategic/ethical alignment (Discernment) defines the Description-Discernment loop — the strategic level of AI Fluency.

66

AI FLUENCY – ASSESSMENT ALIGNMENT

66 A course evaluates students using three instruments: (1) a portfolio of AI-assisted deliverables, (2) weekly prompt-iteration logs, and (3) end-of-module reflections on their decision-making. Which assessment types do these represent in order?

A Process-based, outcome-based, reflection-based.

B **Outcome-based, process-based, reflection-based.** ✓ CORRECT

C Outcome-based, reflection-based, process-based.

D Reflection-based, process-based, outcome-based.

💡 Answer B — Portfolio of deliverables = outcome-based (what was produced). Prompt-iteration logs = process-based (how the student worked). End-of-module reflections = reflection-based (metacognitive awareness).

WRAP-UP

Summary

Context & Escalation 18%

Stateless model, case-facts blocks, lost-in-the-middle, and when to escalate.

Batch API & Extraction 12%

50% cost vs 24h window, custom_id recovery, nullable schemas, retry limits.

Tool Design 21%

Descriptions drive selection; scope per role; typed errors; forced/any tool_choice.

MCP Design Patterns 18%

Project vs user scope, env-var secrets, resources vs tools, hooks for guarantees.

Agentic Systems 12%

Coordinator decomposition, parallel Task spawns, explicit context, crash recovery.

AI Fluency 19%

The 4Ds — Delegation, Description, Discernment, Diligence — and assessment types.

REMEMBER THIS

Key Takeaways

Hard rules need hard enforcement

When a question says "guarantee," "must," or "zero failures," choose programmatic hooks/prerequisite gates — not prompt wording.

Structure your handoffs & errors

Typed error metadata (errorCategory, isRetryable) and compact escalation summaries beat raw transcripts and generic failures.

Practice context hygiene

Trim verbose tool outputs, keep a persistent case-facts block, and counter lost-in-the-middle with summaries + section headers.

Escalate on the right triggers

Explicit human request, policy gap/silence, or no meaningful progress — not sentiment or self-reported confidence.

Tool selection lives in descriptions

Differentiate similar tools, scope each agent to its role, and prefer the proportionate first fix over new ML infrastructure.

Match the API to the latency

Batch API for overnight/latency-tolerant jobs; synchronous API for blocking, interactive, or multi-turn tool-calling work.

BEFORE YOU SIT

Exam Tips

1. **Read the stem's exact ask** — Distinguish "most effective FIRST step" from "BEST/MOST effective" — the proportionate fix often wins over the maximal one.
2. **Prefer determinism on "must"** — If a rule must hold every time (money, identity, ordering), pick hooks/gates over prompt instructions, which fail probabilistically.
3. **Find the real root cause** — In multi-agent failures, blame the agent that made the decision (often the coordinator), not downstream agents working within their scope.
4. **Reject over-engineering** — When prompt/description tweaks haven't been tried, a new classifier or ML pipeline is usually the wrong, heavier answer.
5. **Classify errors precisely** — Separate transient vs validation vs business vs permission, and access failures vs valid empty results, before choosing recovery.
6. **Let stop_reason drive loops** — Continue on "tool_use", stop on "end_turn" — never terminate on text phrases or arbitrary iteration caps.
7. **Ignore answer length & letter** — Correct options here are randomized in position and length — decide on the concept, not the longest or most-qualified-sounding choice.